

Comandos Git

#versionamento

#git

Documentação do Git: <https://git-scm.com/doc>

Livro Pro Git (gratuito, em português): <https://git-scm.com/book/pt-br/v2>

Primeiros Passos

Instalação:

O git está disponível no repositório oficial do arch linux

```
sudo pacman -S git
```

Configuração inicial

Antes de usar o Git é necessário dizer quem você é, pois essa informação vai junto em cada commit que você fizer

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

`--global` : Aplica essa configuração para todos os repositórios do seu usuário. Se quiser configurar algo só pra um repositório específico, basta rodar o comando sem essa flag dentro da pasta do projeto.

Para conferir todas as configurações atuais:

```
git config --list
```

Info

O Git funciona em três "áreas": a **working directory** (seus arquivos sendo editados), a **staging area** (também chamada de índice, onde você prepara o que vai ser commitado) e o **repository** (o histórico de commits salvo). Entender esse fluxo é a chave pra entender os comandos abaixo.

Iniciando um repositório

`git init`

Transforma a pasta atual em um repositório Git, criando a pasta oculta `.git` que guarda todo o histórico

```
git init
```

`-b <nome>` ou `--initial-branch=<nome>` : Define o nome da branch principal já na criação (ex: `git init -b main`), evitando ter que renomear depois

`--bare` : Cria um repositório "nu", sem working directory, usado geralmente em servidores que só armazenam o histórico pra outros clonarem/enviarem código

`git clone`

Copia um repositório remoto inteiro (com todo o histórico) para a sua máquina

```
git clone https://github.com/usuario/repositorio.git
```

`<destino>` : Pode ser passado um segundo argumento com o nome da pasta que vai receber o projeto, senão usa o nome do repositório

`--depth <n>` : Faz um clone "raso", trazendo só os últimos `n` commits em vez do histórico completo (útil pra repositórios grandes e antigos)

`-b <branch>` ou `--branch <branch>` : Já clona direto em cima de uma branch específica, em vez da branch padrão

`--recurse-submodules` : Clona também os submódulos do projeto, caso ele use

Verificando o estado do repositório

`git status`

Mostra o que mudou: arquivos modificados, novos (não rastreados) e o que já está na staging area

```
git status
```

`-s` ou `--short` : Mostra a saída de forma resumida, uma linha por arquivo, com códigos como M (modificado), A (adicionado), ?? (não rastreado)

`-b` : Mostra também informações da branch atual e se ela está à frente/atrás do remoto

git log

Mostra o histórico de commits

```
git log
```

`--oneline` : Resume cada commit em uma linha só (hash curto + mensagem)

`--graph` : Desenha uma árvore em ASCII mostrando branches e merges, muito útil combinado com `--oneline`

`--all` : Mostra o histórico de todas as branches, não só da atual

`-n <número>` : Limita a quantidade de commits exibidos (ex: `git log -n 5`)

`-p` : Mostra o diff completo (o que mudou linha a linha) de cada commit

`--stat` : Mostra um resumo de quantos arquivos e linhas foram alterados em cada commit, sem mostrar o diff inteiro

git diff

Mostra as diferenças entre versões de arquivos, linha a linha

```
git diff
```

Sem argumentos: Compara a working directory com a staging area (ou seja, mudanças ainda não adicionadas com `git add`)

`--staged` ou `--cached` : Compara a staging area com o último commit (ou seja, mostra o que já foi adicionado e será commitado)

`<commit1> <commit2>` : Compara dois commits específicos entre si

Registrando mudanças

git add

Move mudanças da working directory para a staging area, preparando o que vai entrar no próximo commit

```
git add arquivo.py
```

`.` : Adiciona todos os arquivos modificados e novos da pasta atual e subpastas

`-A` ou `--all` : Adiciona todas as mudanças do repositório inteiro, incluindo exclusões de arquivos

`-p` ou `--patch` : Modo interativo que permite escolher pedaço por pedaço (hunk) de um arquivo o que vai ou não pra staging area, útil pra dividir uma mudança grande em commits menores e mais organizados

git commit

Salva permanentemente no histórico tudo o que está na staging area

```
git commit -m "Mensagem descrevendo a mudança"
```

`-m <mensagem>` : Define a mensagem do commit direto na linha de comando, sem abrir o editor de texto padrão

`-a` : Adiciona automaticamente na staging area todos os arquivos já rastreados que foram modificados (não pega arquivos novos), pulando a etapa manual do `git add`. Muito usado combinado como `git commit -am "mensagem"`

`--amend` : Reaproveita o último commit em vez de criar um novo, útil pra corrigir a mensagem ou esquecer de incluir um arquivo. Só deve ser usado em commits que ainda não foram enviados (`push`) pro remoto

git rm

Remove um arquivo tanto da working directory quanto do controle do Git

```
git rm arquivo.py
```

`--cached` : Remove o arquivo apenas do controle do Git (ele deixa de ser rastreado), mas mantém o arquivo físico na pasta. Muito usado quando um arquivo foi commitado por engano e precisa ser adicionado ao `.gitignore`

Info

O arquivo `.gitignore` lista padrões de arquivos e pastas que o Git deve ignorar (não mostrar como não rastreados e nunca adicionar sem forçar). Exemplo de conteúdo:

```
node_modules/  
*.log
```

```
.env  
target/
```

Trabalhando com branches

git branch

Gerencia branches (ramificações do histórico)

```
git branch nova-feature
```

Sem argumentos: Lista as branches locais, marcando com * a branch atual

-a : Lista também as branches remotas conhecidas

-d <branch> : Deleta uma branch local (só funciona se ela já foi mesclada em outra, evitando perda de trabalho)

-D <branch> : Força a deleção de uma branch local mesmo que ela tenha mudanças não mescladas

-m <antigo> <novo> : Renomeia uma branch

git switch

Troca a branch atual (comando mais novo e recomendado, criado pra separar essa função do checkout)

```
git switch nova-feature
```

-c <nome> : Cria a branch e já troca pra ela em um único comando (equivalente ao antigo checkout -b)

git checkout

Comando mais antigo e "multiuso": troca de branch, mas também restaura arquivos

```
git checkout nova-feature
```

-b <nome> : Cria uma nova branch a partir da atual e já troca pra ela

-- <arquivo> : Descarta as mudanças não commitadas de um arquivo específico, voltando ele pra como estava no último commit (ex: git checkout -- arquivo.py)

git merge

Junta o histórico de outra branch na branch atual

```
git merge nova-feature
```

`--no-ff` : Força a criação de um commit de merge mesmo quando seria possível um "fast-forward" (avanço direto do ponteiro), preservando no histórico o fato de que existiu uma branch separada

`--squash` : Junta todos os commits da branch em um único commit "achatado" na branch atual, sem trazer o histórico individual

Info

Quando duas branches alteram a mesma linha de um mesmo arquivo de formas diferentes, acontece um **conflito de merge**. O Git marca o trecho no arquivo com `<<<<<<<`, `=====` e `>>>>>>>`, e é preciso editar manualmente, escolher a versão correta e depois rodar `git add + git commit` pra finalizar o merge.

Trabalhando com repositórios remotos

git remote

Gerencia as conexões com repositórios remotos (ex: GitHub, GitLab)

```
git remote add origin https://github.com/usuario/repositorio.git
```

`add <nome> <url>` : Adiciona um novo remoto com um apelido (por convenção o principal se chama `origin`)

`-v` : Lista os remotos configurados junto com suas URLs

`remove <nome>` : Remove a conexão com um remoto

git push

Envia os commits locais para um repositório remoto

```
git push origin main
```

`origin` : Nome do remoto de destino

`main` : Nome da branch de destino

`-u` ou `--set-upstream` : Associa a branch local à branch remota, permitindo que nas próximas vezes baste rodar `git push` sem especificar remoto e branch

`--force` : Sobrescreve o histórico remoto com o local, mesmo que diverjam. É perigoso porque pode apagar commits de outras pessoas

`--force-with-lease` : Versão mais segura do `--force` , que falha se alguém enviou commits novos pro remoto desde a última vez que você sincronizou, evitando sobrescrever trabalho alheio sem perceber

`--tags` : Envia também as tags criadas localmente, que por padrão não vão junto no push comum

git pull

Atualiza o repositório local trazendo e já mesclando as mudanças do remoto (equivale a um `git fetch` seguido de um `git merge`)

```
git pull origin main
```

`--rebase` : Em vez de criar um commit de merge, reaplica os commits locais por cima dos commits novos que vieram do remoto, deixando o histórico mais linear

git fetch

Baixa as mudanças do remoto, mas sem mesclar nada na branch atual, apenas atualizando as referências (ex: `origin/main`)

```
git fetch origin
```

Info

`git fetch` é mais seguro que `git pull` pra quem quer primeiro ver o que mudou (com `git log origin/main` ou `git diff main origin/main`) antes de decidir mesclar

Desfazendo mudanças

git stash

Guarda temporariamente mudanças não commitadas, limpando a working directory sem precisar commitar nada

`git stash`

`push -m "mensagem"` : Guarda o estado atual com uma mensagem descritiva (a partir de versões mais novas do Git, `git stash` sozinho já equivale a `git stash push`)

`list` : Lista todos os stashes salvos

`pop` : Aplica o stash mais recente de volta na `working directory` e o remove da lista

`apply` : Aplica o stash mais recente de volta, mas sem removê-lo da lista (útil pra aplicar o mesmo stash em mais de uma branch)

`drop` : Remove um stash da lista sem aplicá-lo

`git reset`

Move o ponteiro da branch atual para outro commit, podendo desfazer commits

```
git reset --mixed HEAD~1
```

`--soft` : Move o ponteiro do commit, mas mantém as mudanças na staging area (como se o commit nunca tivesse acontecido, mas o `git add` sim)

`--mixed` (padrão): Move o ponteiro e desfaz a staging area, mas mantém as mudanças na `working directory`

`--hard` : Move o ponteiro e descarta completamente as mudanças, tanto da staging area quanto da `working directory`. **Perigoso**, pois apaga o trabalho de forma irreversível

`HEAD~1` : Notação que indica "um commit antes do atual"; pode-se usar `HEAD~2` , `HEAD~3` etc, ou o hash direto de um commit

`git revert`

Desfaz as mudanças de um commit específico criando um **novo** commit com o efeito contrário, em vez de apagar histórico

```
git revert <hash-do-commit>
```

`--no-commit` : Aplica a reversão na staging area, mas não cria o commit automaticamente, permitindo revisar antes

`git revert` é preferível ao `git reset --hard` quando o commit já foi enviado (`push`) pro remoto e outras pessoas podem ter baixado esse histórico, já que ele não reescreve commits existentes, apenas adiciona um novo.

Fluxo de trabalho típico

Um ciclo comum de uso, juntando os comandos acima:

```
git switch -c minha-feature
# ... edita os arquivos ...
git add .
git commit -m "Implementa minha feature"
git push -u origin minha-feature
```

Depois, ao voltar pro projeto em outro dia, é comum atualizar a branch principal antes de continuar:

```
git switch main
git pull origin main
git switch minha-feature
git merge main
```

Info

Assim como no Docker, onde cada contêiner é uma instância isolada de uma imagem, no Git cada branch é uma "linha do tempo" isolada de commits, que depois pode ser mesclada de volta na principal através do `merge` .